

TERRAFORM – TASK – 04

2. Execute the script shown in the video.

Open **VS Code** and create a folder like **Terraform Basic**

Create file:

main.tf

In main.tf, write Terraform provider + local file resource

Open **Terminal** inside VS Code

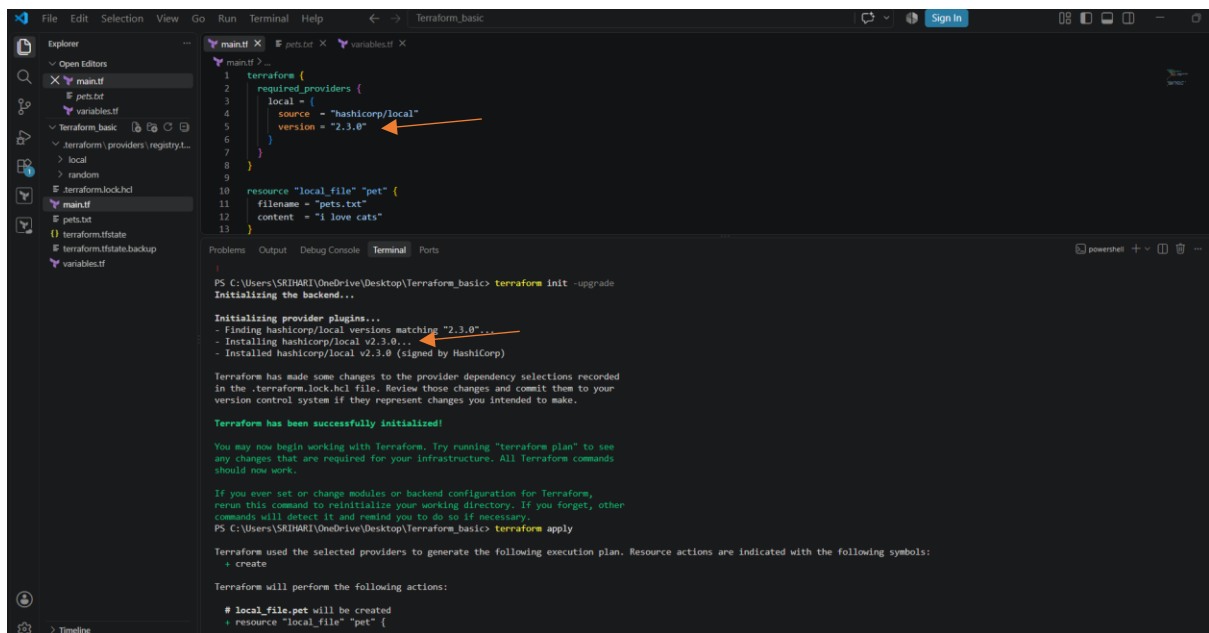
Open main.tf

```
terraform {  
  required_providers {  
    local = {  
      source = "hashicorp/local"  
      version = "2.3.0"  
    }  
  }  
}  
  
resource "local_file" "pet" {  
  filename = "pets.txt"  
  content = "i love cats"  
}
```

Run:

terraform init

It will download the provider plugin and show **Terraform has been successfully initialized**



Open main.tf

Add this code:

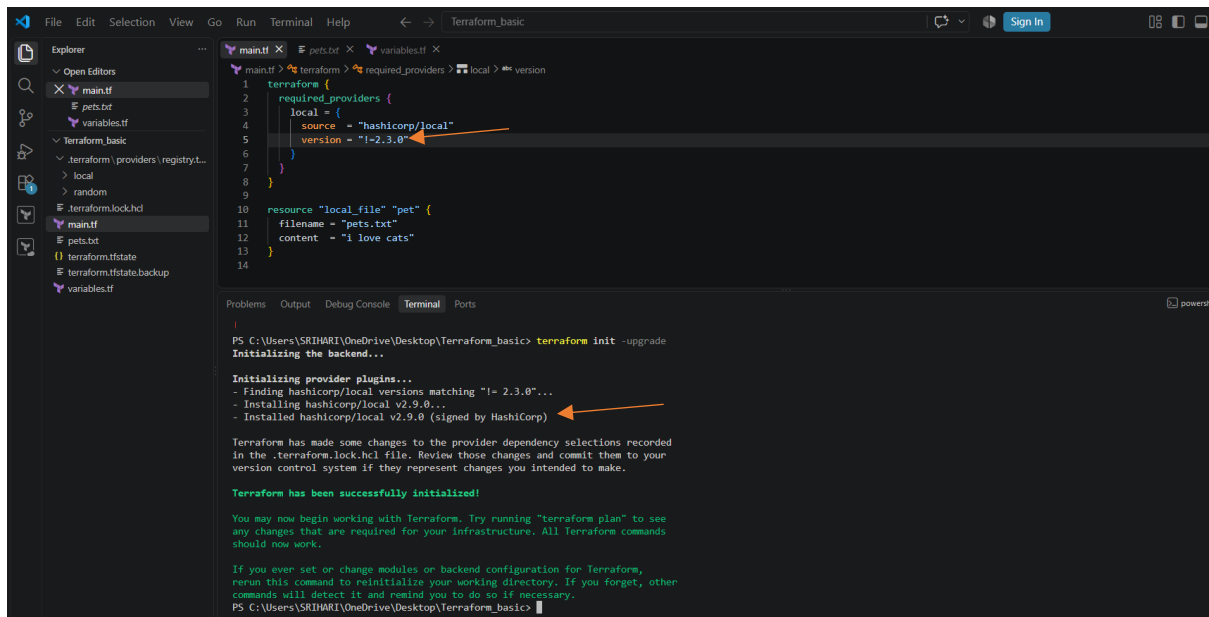
```
terraform {
  required_providers {
    local = {
      source = "hashicorp/local"
      version = "2.3.0"
    }
  }
}
```

Save the file

Run:

terraform init

It installs the required local provider **2.3.0**



Change version to:

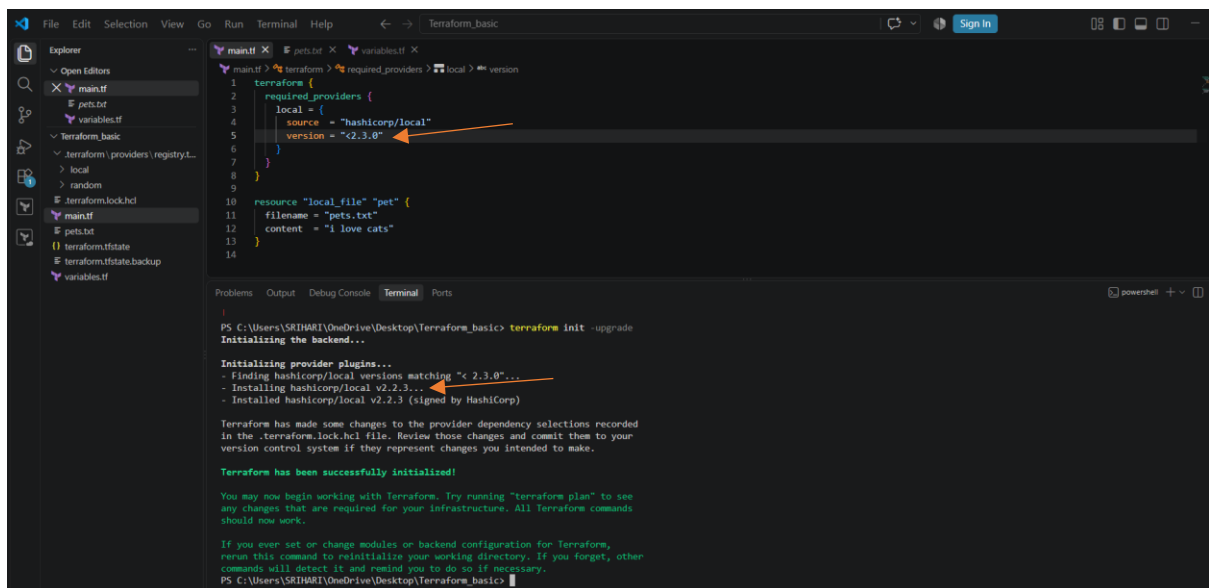
version = "< 2.3.0"

Save the file

Run:

terraform init

Terraform matches and installs a provider version **less than 2.3.0**



Update version:

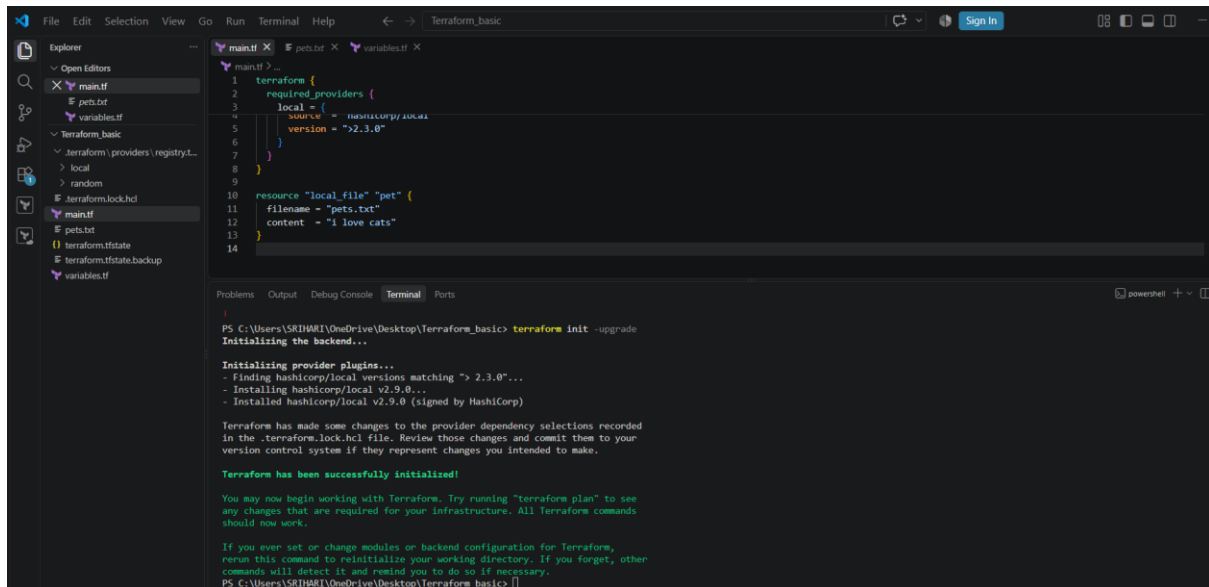
version = "> 2.3.0"

Save the file

Run:

terraform init

It installs the version **greater than 2.3.0** (latest possible)



The screenshot shows the VS Code interface with a file explorer on the left and a terminal window at the bottom. The file explorer shows a project structure with files like main.tf, pets.txt, variables.tf, terraform.lock.hcl, and terraform.tfstate. The terminal window shows the output of the command `terraform init -upgrade`. The output indicates that Terraform is initializing the backend and installing provider plugins. It shows that it found hashicorp/local versions matching `"> 2.3.0"` and installed hashicorp/local v2.9.0 (signed by HashiCorp). The output also mentions that Terraform has made some changes to the provider dependency selections recorded in the `.terraform.lock.hcl` file. The output concludes with "Terraform has been successfully initialized!" and provides instructions on how to begin working with Terraform.

```
PS C:\Users\SRIHARI\OneDrive\Desktop\Terraform_basic> terraform init -upgrade
Initializing the backend...

Initializing provider plugins...
- Finding hashicorp/local versions matching "> 2.3.0"...
- Installing hashicorp/local v2.9.0...
- Installed hashicorp/local v2.9.0 (signed by HashiCorp)

Terraform has made some changes to the provider dependency selections recorded
in the .terraform.lock.hcl file. Review those changes and commit them to your
version control system if they represent changes you intended to make.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
PS C:\Users\SRIHARI\OneDrive\Desktop\Terraform_basic> []
```

Change version like this:

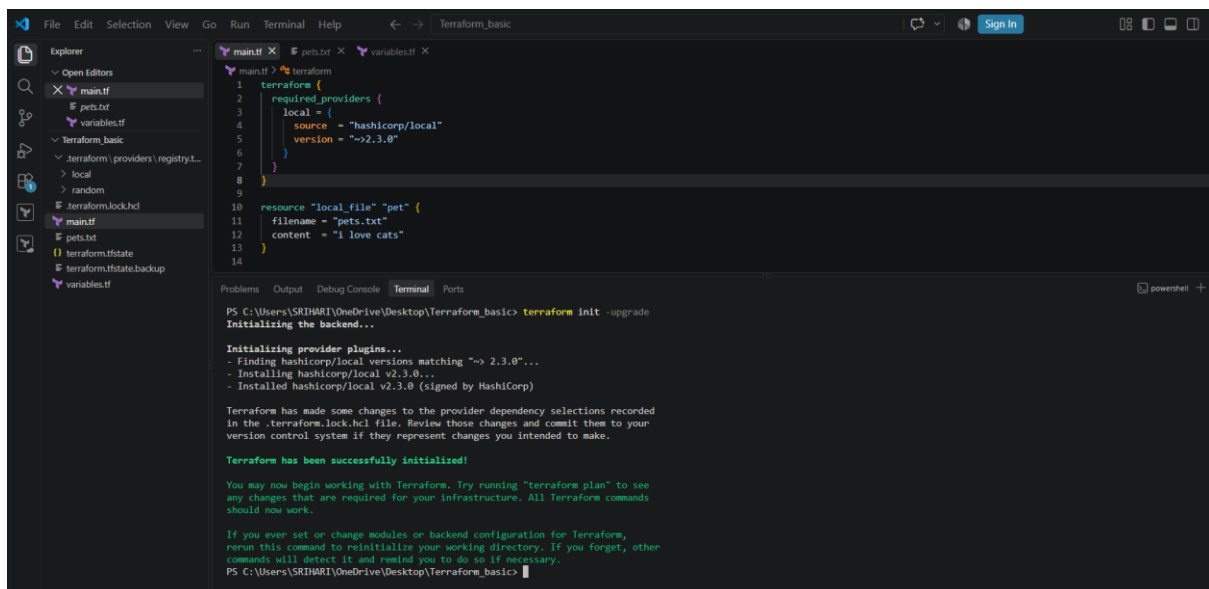
version = "> 2.3.0"

Save the file

Run:

terraform init

Terraform will install **2.3.x versions only** (patch updates allowed)



The screenshot shows the VS Code interface with a file explorer on the left and a terminal window at the bottom. The file explorer shows a project structure with files like main.tf, pets.txt, variables.tf, terraform.lock.hcl, and terraform.tfstate. The terminal window shows the output of the command `terraform init -upgrade`. The output indicates that Terraform is initializing the backend and installing provider plugins. It shows that it found hashicorp/local versions matching `"> 2.3.0"` and installed hashicorp/local v2.3.0 (signed by HashiCorp). The output also mentions that Terraform has made some changes to the provider dependency selections recorded in the `.terraform.lock.hcl` file. The output concludes with "Terraform has been successfully initialized!" and provides instructions on how to begin working with Terraform.

```
PS C:\Users\SRIHARI\OneDrive\Desktop\Terraform_basic> terraform init -upgrade
Initializing the backend...

Initializing provider plugins...
- Finding hashicorp/local versions matching "> 2.3.0"...
- Installing hashicorp/local v2.3.0...
- Installed hashicorp/local v2.3.0 (signed by HashiCorp)

Terraform has made some changes to the provider dependency selections recorded
in the .terraform.lock.hcl file. Review those changes and commit them to your
version control system if they represent changes you intended to make.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
PS C:\Users\SRIHARI\OneDrive\Desktop\Terraform_basic> []
```

In main.tf, used a data source:

```
data "local_file" "pet" {  
  filename = "pets.txt"  
}
```

Run:

terraform apply

Terraform shows error:

Read local file data source error

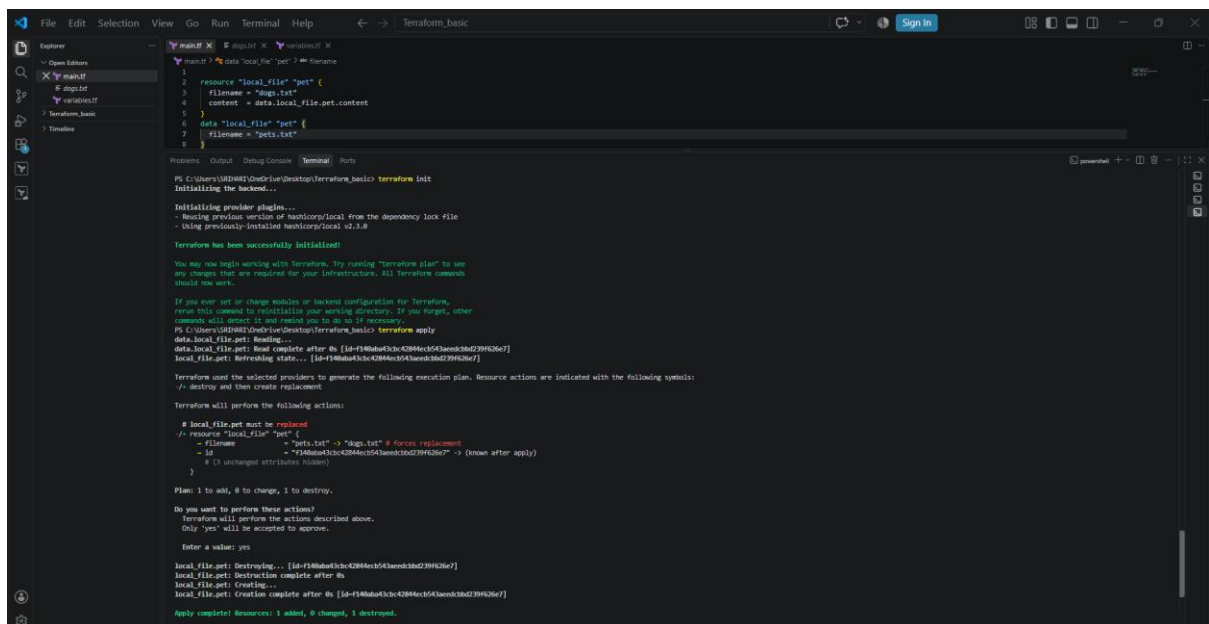
I have pets.txt file in my local

Content = I love cats

Run:

terraform apply

Now the data source reads successfully and resource gets created



```
PS C:\Users\GIRISH\Desktop\terraform_basics> terraform init  
Initializing the backend...  
  
Initializing provider plugins...  
- Reusing previous version of hashicorp/local from the dependency lock file  
- Using previously-installed hashicorp/local v2.3.8  
  
Terraform has been successfully initialized!  
  
You may now begin working with Terraform. Try running "terraform plan" to see  
any changes that are required for your infrastructure. All Terraform commands  
should now work.  
  
If you ever set or change modules or backend configuration for Terraform,  
run this command to reinitialize your working directory. If you forget, other  
commands will detect it and remind you to do so if necessary.  
PS C:\Users\GIRISH\Desktop\terraform_basics> terraform apply  
data.local_file.pet: Reading...  
data.local_file.pet: Read complete after 0s [[id-f14b0a03bc42844cc5643a0dc3b0239f636c7]  
local_file.pet: Refreshing state... [[id-f14b0a03bc42844cc5643a0dc3b0239f636c7]  
  
Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:  
- /- destroy and then create replacement  
  
Terraform will perform the following actions:  
  
# local_file.pet must be replaced  
-/ resource "local_file" "pet" {  
  - filename      = "pets.txt" > "dogs.txt" & forces replacement  
  - id            = "f14b0a03bc42844cc5643a0dc3b0239f636c7" -> (known after apply)  
  - id            = id  
  # (3 unchanged attributes hidden)  
}  
  
Plan: 1 to add, 0 to change, 1 to destroy.  
  
Do you want to perform these actions?  
Terraform will perform the actions described above.  
Only 'yes' will be accepted to approve.  
  
Enter a value: yes  
local_file.pet: Destroying... [[id-f14b0a03bc42844cc5643a0dc3b0239f636c7]  
local_file.pet: Destruction complete after 0s  
local_file.pet: Creating...  
local_file.pet: Creation complete after 0s [[id-f14b0a03bc42844cc5643a0dc3b0239f636c7]  
  
Apply complete! Resources: 1 added, 0 changed, 1 destroyed.
```

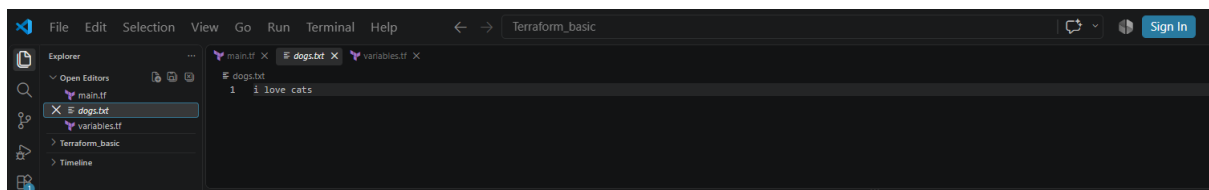
Data replaced to dogs.txt

Open dogs.txt in VS Code

You can see the content:

I love cats

This confirms file was created correctly



```
1 I love cats
```

In main.tf, write:

```
resource "local_file" "file" {  
  
    count  = 3  
  
    filename = "file-${count.index}.txt"  
  
    content = "Hello hari"  
  
}
```

Run:

terraform apply

Terraform will create:

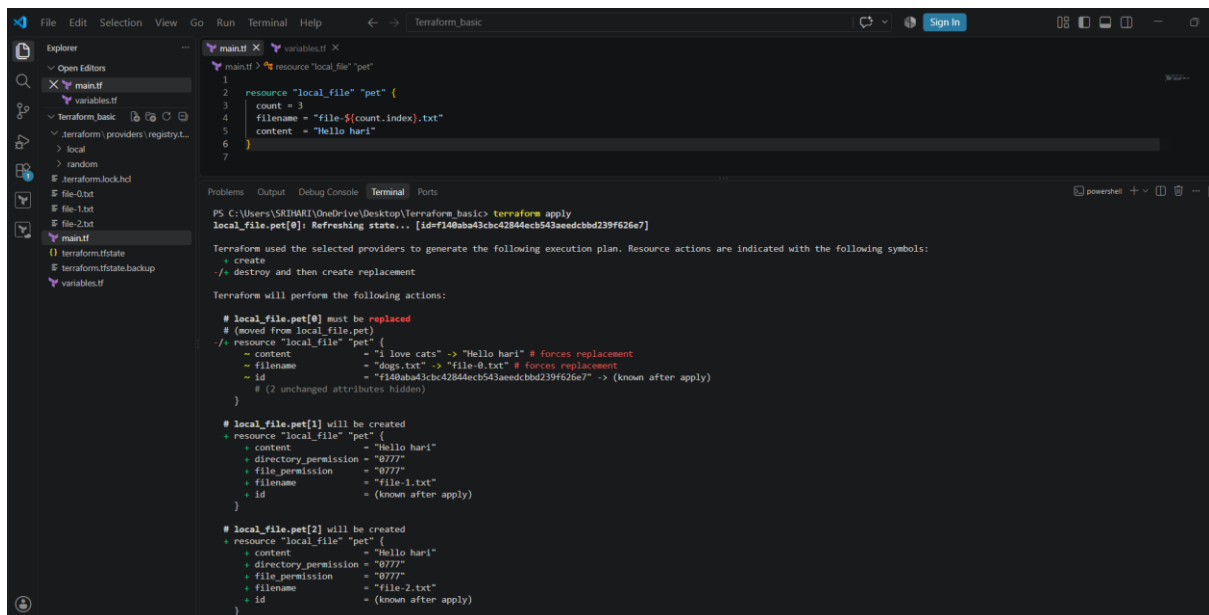
file-0.txt

file-1.txt

file-2.txt

Each file will contain:

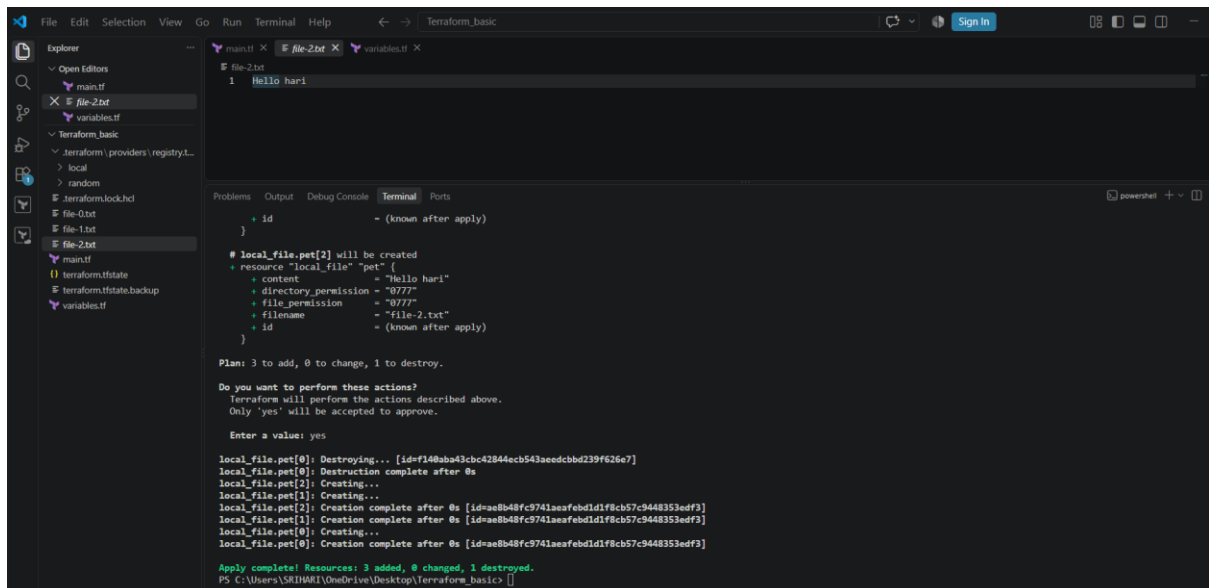
Hello hari



The screenshot shows a Visual Studio Code editor with a Terraform configuration file named `main.tf` open. The configuration defines a `local_file` resource named `pet` with a `count` of 3, a `filename` pattern of `file-${count.index}.txt`, and a `content` of `"Hello hari"`. The file explorer on the left shows the project structure, including `main.tf`, `variables.tf`, and the generated files `file-0.txt`, `file-1.txt`, and `file-2.txt`.

The terminal window displays the output of the `terraform apply` command. It shows the execution plan, indicating that the `local_file.pet` resource must be replaced. The plan details the actions for each instance of the resource, including the creation of `file-0.txt`, `file-1.txt`, and `file-2.txt`, all with the content `"Hello hari"`.

```
PS C:\Users\SRINARI\OneDrive\Desktop\Terraform_basics> terraform apply  
local_file.pet[0]: #refreshing state... [id=f140aba43cbc42844ecb543aeedcbdd239f626e7]  
  
Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:  
+ create  
-/+ destroy and then create replacement  
  
Terraform will perform the following actions:  
  
# local_file.pet[0] must be replaced  
# (moved from local_file.pet)  
/+ resource "local_file" "pet" {  
  ~ content      = "i love cats" -> "Hello hari" # forces replacement  
  ~ filename     = "dogs.txt" -> "file-0.txt" # forces replacement  
  ~ id           = "f140aba43cbc42844ecb543aeedcbdd239f626e7" -> (known after apply)  
  # (2 unchanged attributes hidden)  
}  
  
# local_file.pet[1] will be created  
+ resource "local_file" "pet" {  
  + content      = "Hello hari"  
  + directory_permission = "0777"  
  + file_permission  = "0777"  
  + filename       = "file-1.txt"  
  + id            = (known after apply)  
}  
  
# local_file.pet[2] will be created  
+ resource "local_file" "pet" {  
  + content      = "Hello hari"  
  + directory_permission = "0777"  
  + file_permission  = "0777"  
  + filename       = "file-2.txt"  
  + id            = (known after apply)  
}
```



COUNT:

Open main.tf

Write this code:

```
resource "local_file" "pet" {

  filename = var.filename[count.index]

  count   = length(var.filename)

  content = "my name is hari"
}
```

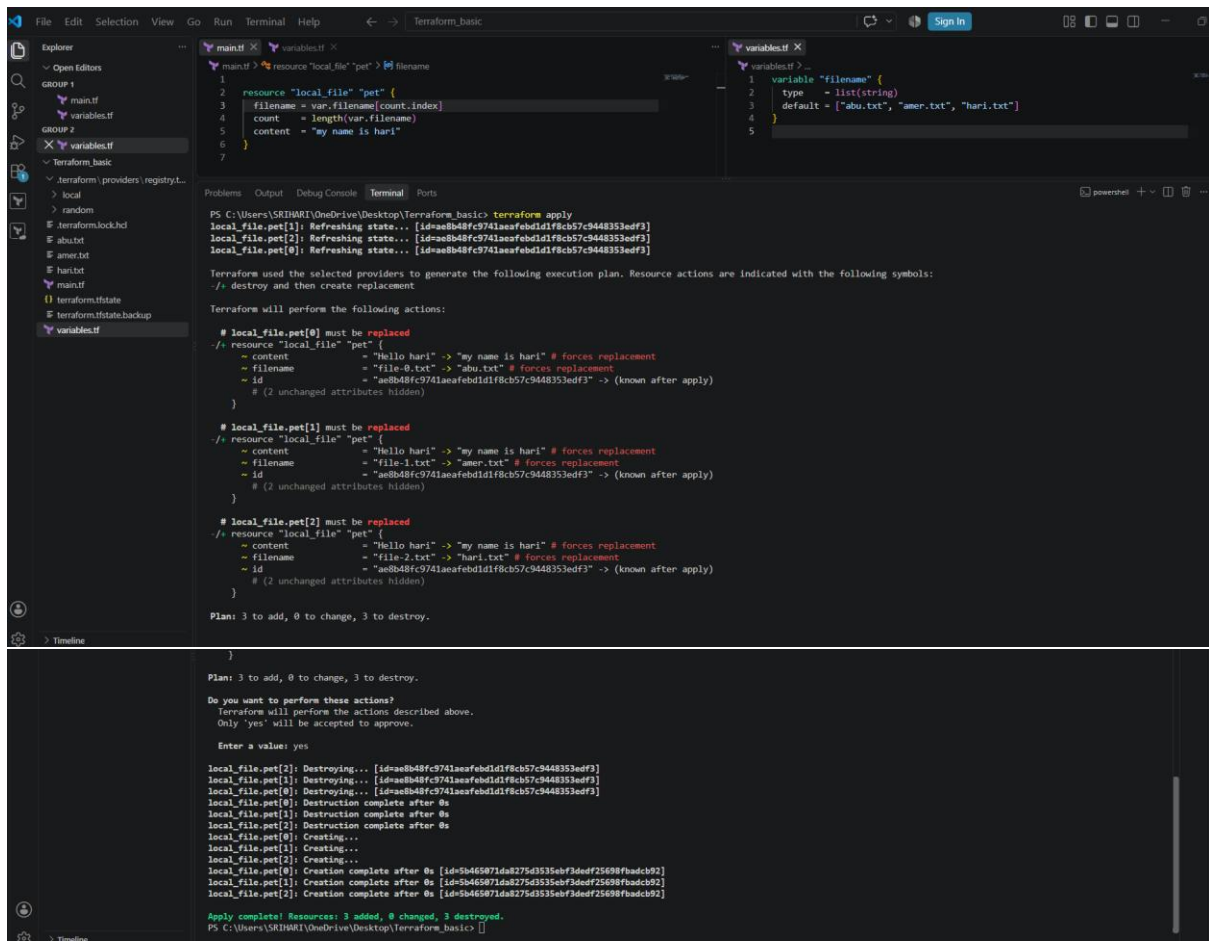
Open variable.tf

```
variable "filename" {

  type    = list(string)

  default = ["abu.txt", "amer.txt", "hari.txt"]

}
```



for_each :

Open main.tf

Write for_each block:

```
resource "local_file" "pet" {

  filename = each.value

  content = "my name is ${each.value}"

  for_each = toset(var.filename)
}
```

Open variable.tf

```
variable "filename" {

  type = list(string)

  default = ["abu.txt", "amer.txt", "hari.txt"]

}
```

Save files

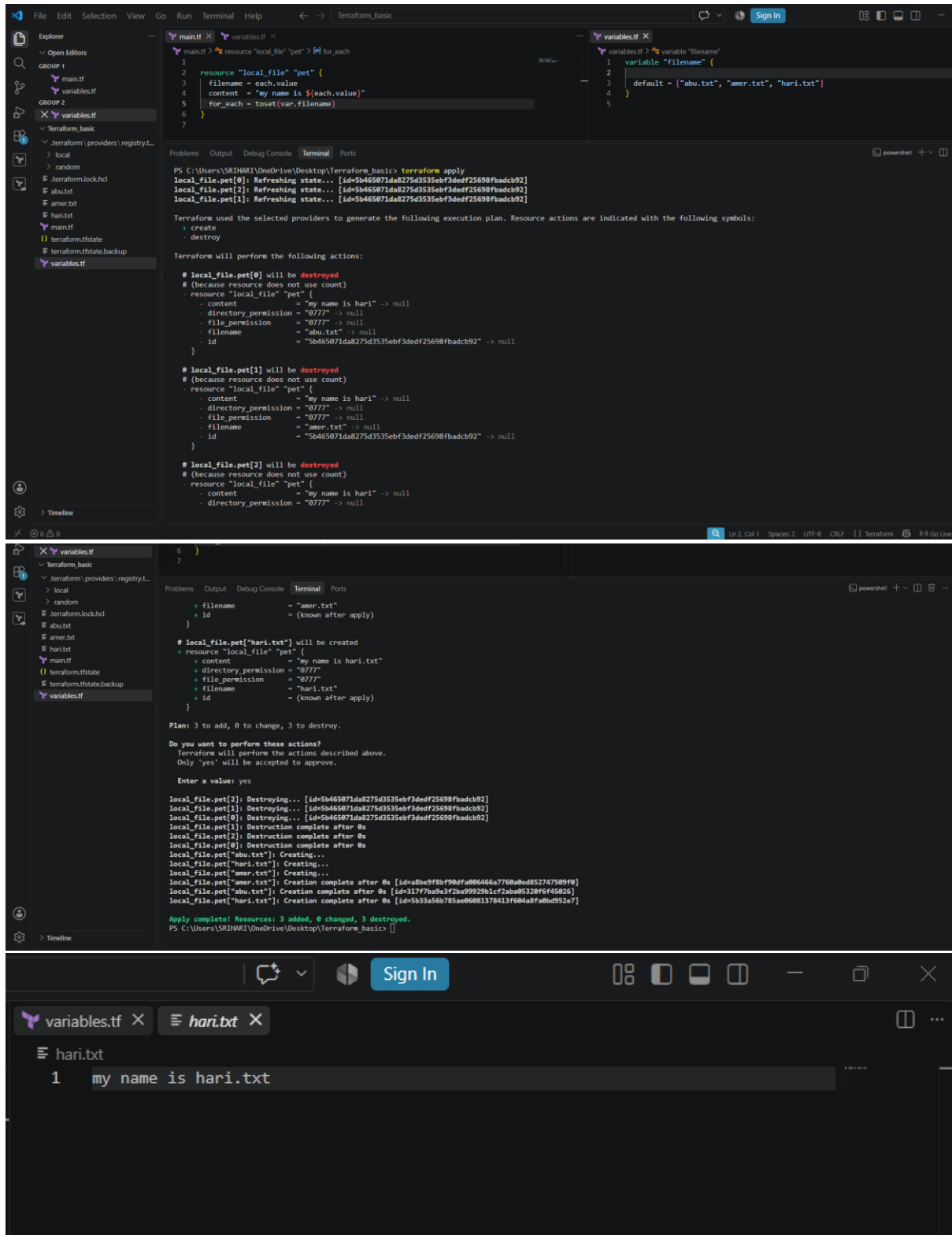
Run:

terraform apply

Terraform starts refreshing state for older resources

It shows new Plan:

It displays filename and content values from variables



Login to AWS Console

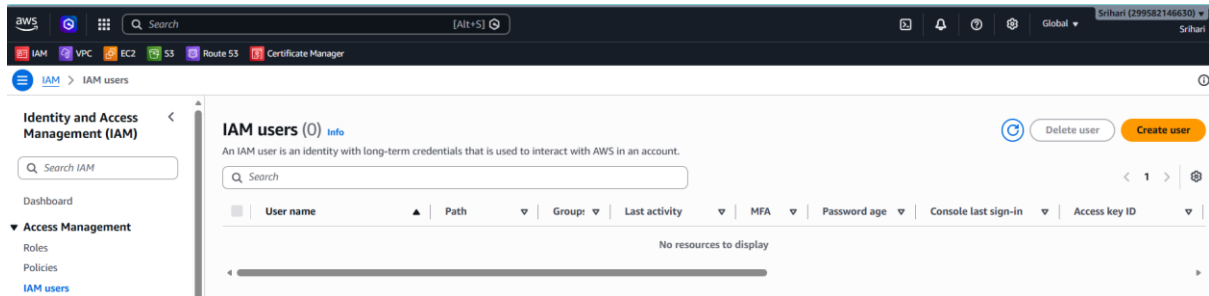
Go to:

IAM → Users

It shows:

Users (0)

This means no IAM users are created yet in that AWS account



Write IAM User Resource in main.tf

Add code:

```
resource "aws_iam_user" "admin_user" {  
  name = "ironman"  
  
  tags = {  
    description = "Technical Team Lead"  
  }  
}
```

Step 2: Configure AWS CLI Credentials

Run:

aws configure

Enter:

AWS Access Key ID

AWS Secret Access Key

Region (example: ap-south-1)

Output format: json

Step 3: Run Terraform Apply (shows lock file issue)

Run:

terraform apply

got error:

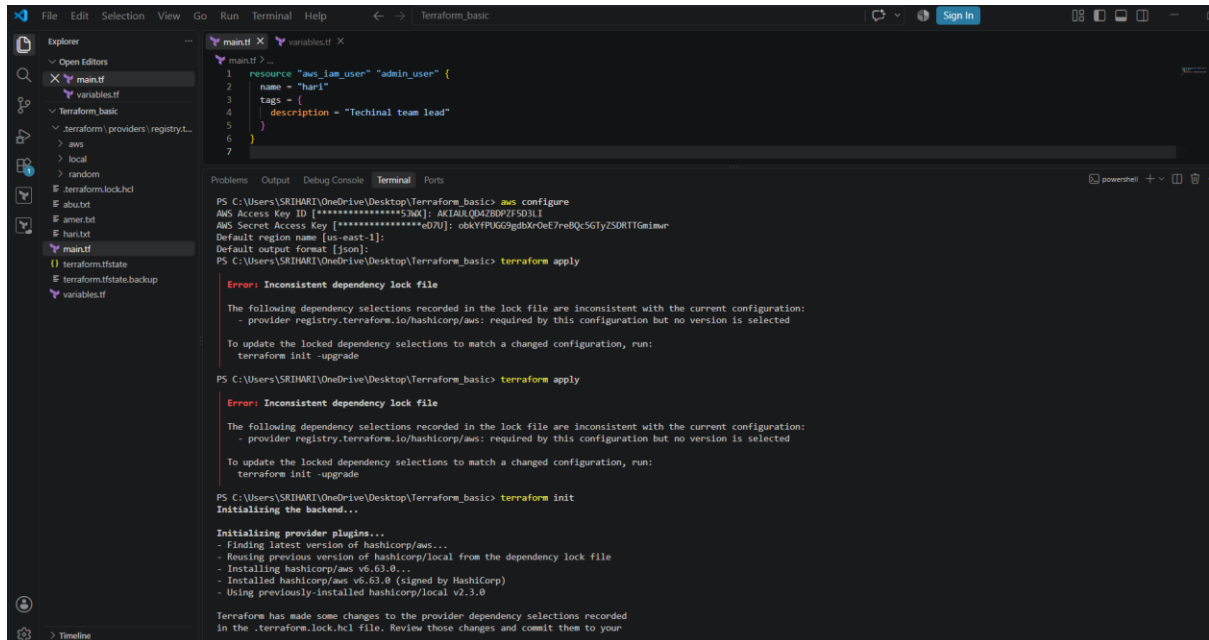
Inconsistent dependency lock file

Step 4: Fix Error using Terraform Init

Run:

terraform init

It downloads AWS provider and resolves lock file mismatch



```
File Edit Selection View Go Run Terminal Help
main.tf X variables.tf X
main.tf X
variables.tf X
Terraform\providers\registry...
aws
local
random
Terraform.lock.hcl
about.txt
amer.txt
har.txt
main.tf
terraform.tfstate
terraform.tfstate.backup
variables.tf

PS C:\Users\SRIHARI\OneDrive\Desktop\Terraform_basics> aws configure
AWS Access Key ID [*****@*****]: AKIAQQA2ZD9P1FSDLI
AWS Secret Access Key [*****@*****]: obkYFPXG6gdbXc0e7reBQ:SGTyZS0RTTGmimw
Default region name [us-east-1]:
Default output format [json]:
PS C:\Users\SRIHARI\OneDrive\Desktop\Terraform_basics> terraform apply
Error: Inconsistent dependency lock file
The following dependency selections recorded in the lock file are inconsistent with the current configuration:
- provider registry.terraform.io/hashicorp/aws: required by this configuration but no version is selected
To update the locked dependency selections to match a changed configuration, run:
terraform init -upgrade
PS C:\Users\SRIHARI\OneDrive\Desktop\Terraform_basics> terraform apply
Error: Inconsistent dependency lock file
The following dependency selections recorded in the lock file are inconsistent with the current configuration:
- provider registry.terraform.io/hashicorp/aws: required by this configuration but no version is selected
To update the locked dependency selections to match a changed configuration, run:
terraform init -upgrade
PS C:\Users\SRIHARI\OneDrive\Desktop\Terraform_basics> terraform init
Initializing the backend...

Initializing provider plugins...
- Finding latest version of hashicorp/aws...
- Reusing previous version of hashicorp/local from the dependency lock file
- Installing hashicorp/aws v6.63.0...
- Installed hashicorp/aws v6.63.0 (signed by HashiCorp)
- Using previously-installed hashicorp/local v2.3.0

Terraform has made some changes to the provider dependency selections recorded
in the .terraform.lock.hcl file. Review those changes and commit them to your
```

Run:

terraform apply

Terraform displays execution plan:

aws_iam_user.admin_user will be created

It shows:

username = ironman

tags.description = Technical Team Lead

Enter:

yes

Terraform creates the IAM user successfully

```
1 resource "aws_iam_user" "admin_user" {
2   name = "hari"
3   tags = {
4     description = "Techinal team lead"
5   }
6 }
7
```

Problems Output Debug Console Terminal Ports

- Installing hashicorp/aws v6.63.0...
- Installed hashicorp/aws v6.63.0 (signed by HashiCorp)
- Using previously-installed hashicorp/local v2.3.8

Terraform has made some changes to the provider dependency selections recorded in the .terraform.lock.hcl file. Review those changes and commit them to your version control system if they represent changes you intended to make.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see any changes that are required for your infrastructure. All Terraform commands should now work.

If you ever set or change modules or backend configuration for Terraform, rerun this command to reinitialize your working directory. If you forget, other commands will detect it and remind you to do so if necessary.

PS C:\Users\SRIDHARI\OneDrive\Desktop\Terraform_basic> terraform apply
local_file.pet["hari.txt"]: Refreshing state... [id=5b33a56b785ae06081378413f604a8fa0bd952e7]
local_file.pet["amer.txt"]: Refreshing state... [id=a8be9f8bf90efaf086466a7760a0ed852747509f8]
local_file.pet["abu.txt"]: Refreshing state... [id=31777b0e3f2ba99929b1cf2aba05320f6f45026]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create
- destroy

Terraform will perform the following actions:

```
# aws_iam_user.admin_user will be created
+ resource "aws_iam_user" "admin_user" {
+   aws           = (known after apply)
+   force_destroy = false
+   id            = (known after apply)
+   name          = "hari"
+   path          = "/"
+   tags          = {
+     "description" = "Techinal team lead"
  }
```

Plan: 1 to add, 0 to change, 3 to destroy.

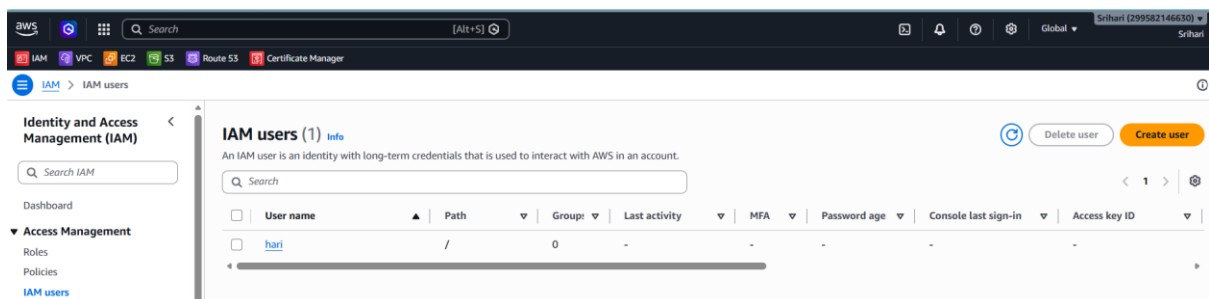
Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

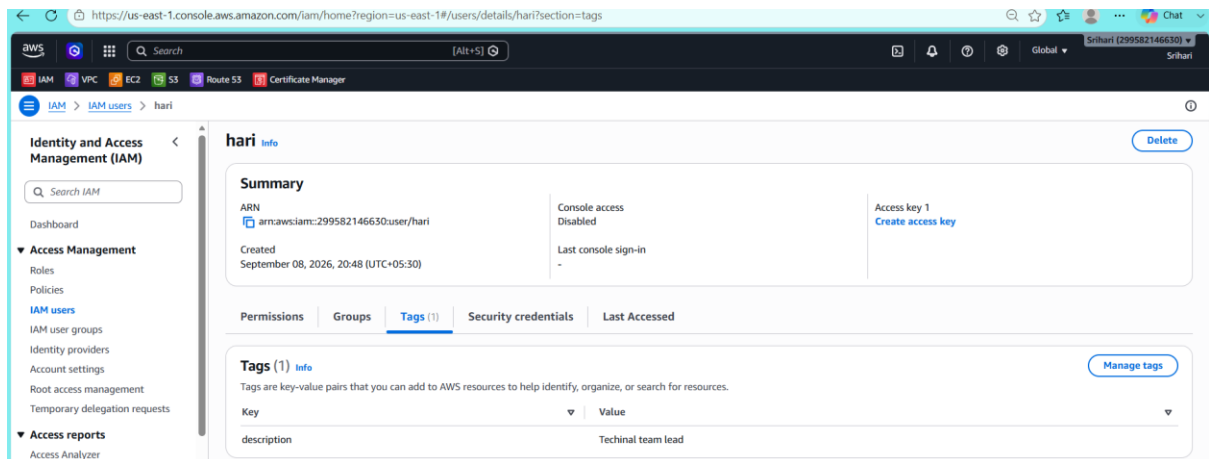
local_file.pet["abu.txt"]: Destroying... [id=31777b0e3f2ba99929b1cf2aba05320f6f45026]
local_file.pet["hari.txt"]: Destroying... [id=5b33a56b785ae06081378413f604a8fa0bd952e7]
local_file.pet["amer.txt"]: Destroying... [id=a8be9f8bf90efaf086466a7760a0ed852747509f8]
local_file.pet["hari.txt"]: Destruction complete after 0s
local_file.pet["abu.txt"]: Destruction complete after 0s
local_file.pet["amer.txt"]: Destruction complete after 0s
aws_iam_user.admin_user: Creation complete after 2s [id=hari]

Apply complete! Resources: 1 added, 0 changed, 3 destroyed.
PS C:\Users\SRIDHARI\OneDrive\Desktop\Terraform_basic>

User added successfully



TAG:



Login to **AWS Console**

Go to:

IAM → Users

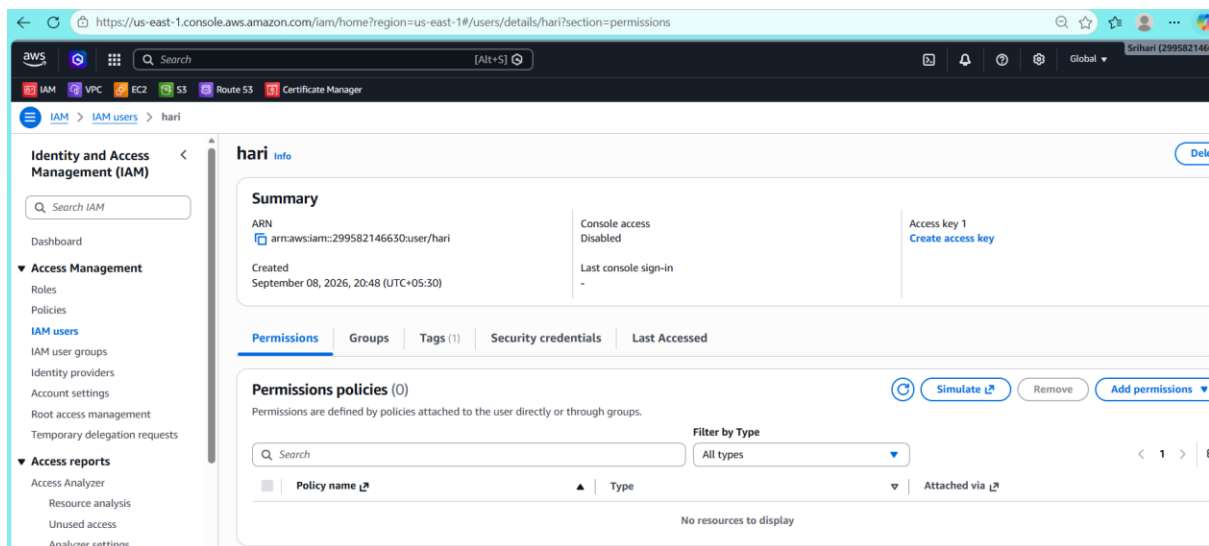
Open the user **ironman**

Click **Permissions** tab

You can see:

Permissions policies (0)

Meaning: **No policy is attached yet**, so user has no access permissions.



Open **main.tf** in VS Code

Add IAM user resource:

```
resource "aws_iam_user" "admin_user" {  
  name = "srihari"  
  
  tags = {  
    description = "Technical Team Lead"  
  }  
}
```

```
}  
}
```

Add IAM policy resource:

```
resource "aws_iam_policy" "adminuser" {  
  name = "AdminUsers"
```

```
  
  policy = <<EOF
```

```
{  
  "Version": "2012-10-17",  
  "Statement": [  
    {  
      "Sid": "1234567890",  
      "Effect": "Allow",  
      "Action": "*",  
      "Resource": "*"   
    }  
  ]  
}
```

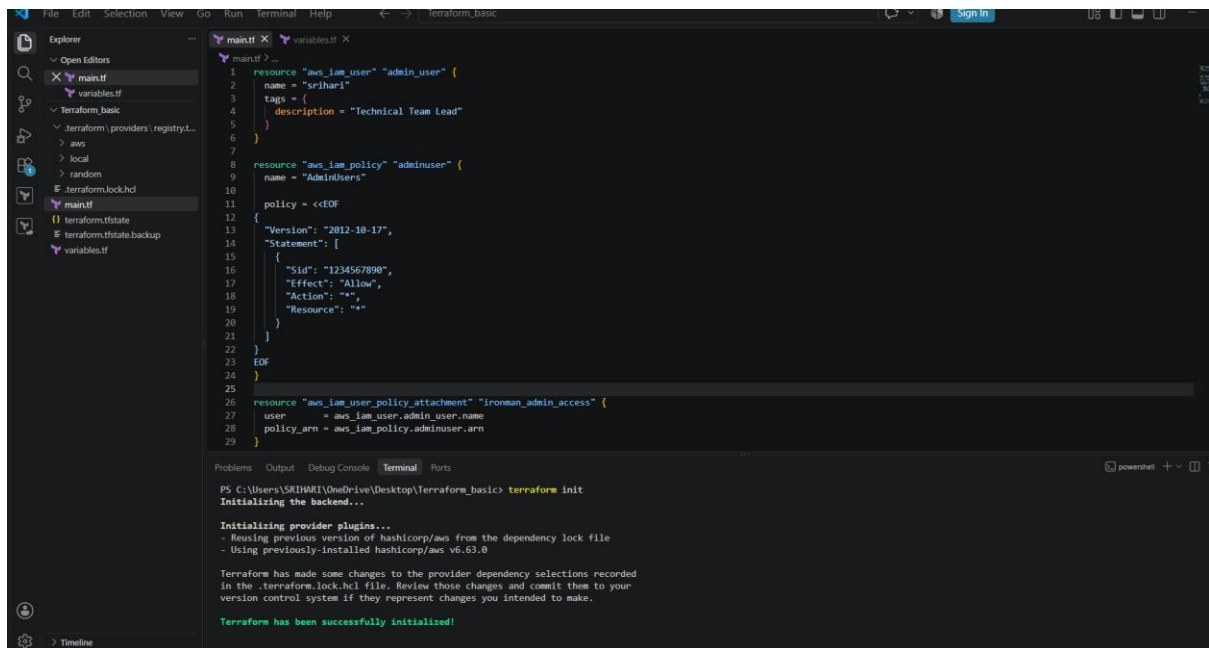
```
EOF
```

```
}
```

Attach the policy to the user:

```
resource "aws_iam_user_policy_attachment" "ironman_admin_access" {  
  user      = aws_iam_user.admin_user.name  
  policy_arn = aws_iam_policy.adminuser.arn  
}
```

Save the file



Open terminal in VS Code

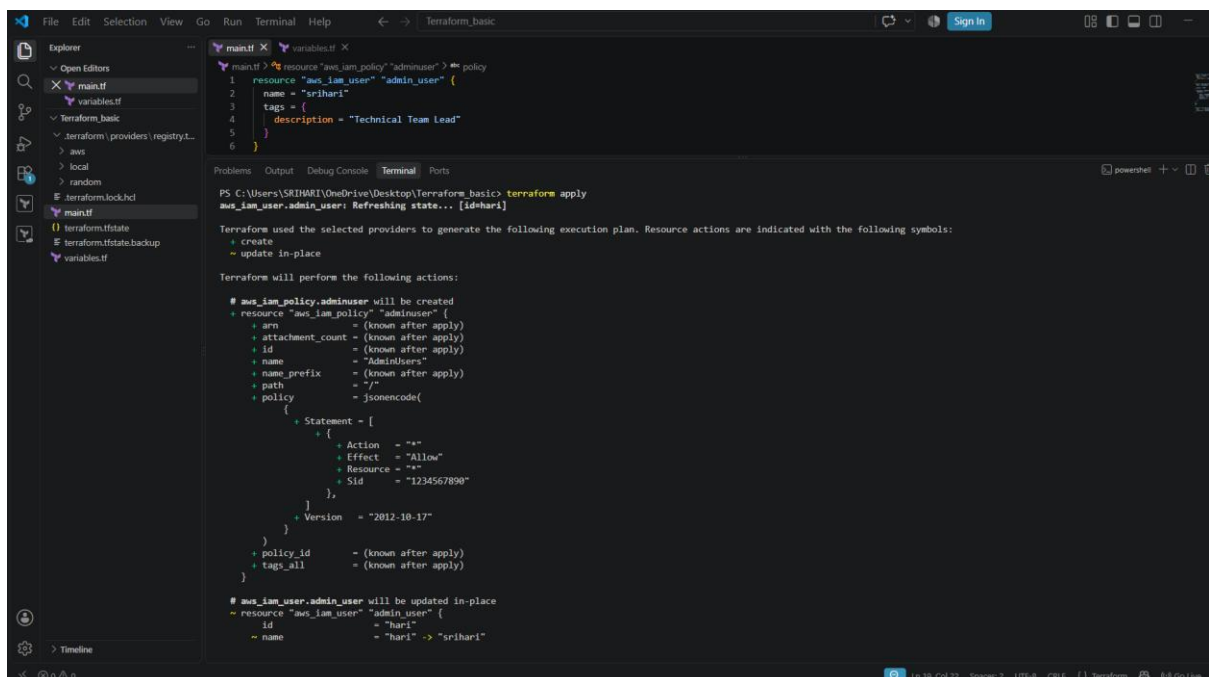
Run:

terraform apply

Terraform shows plan output:

aws_iam_policy.adminuser will be created

It displays the JSON policy in the plan (Allow "Action": "*")



When Terraform asks confirmation:

Enter a value: yes

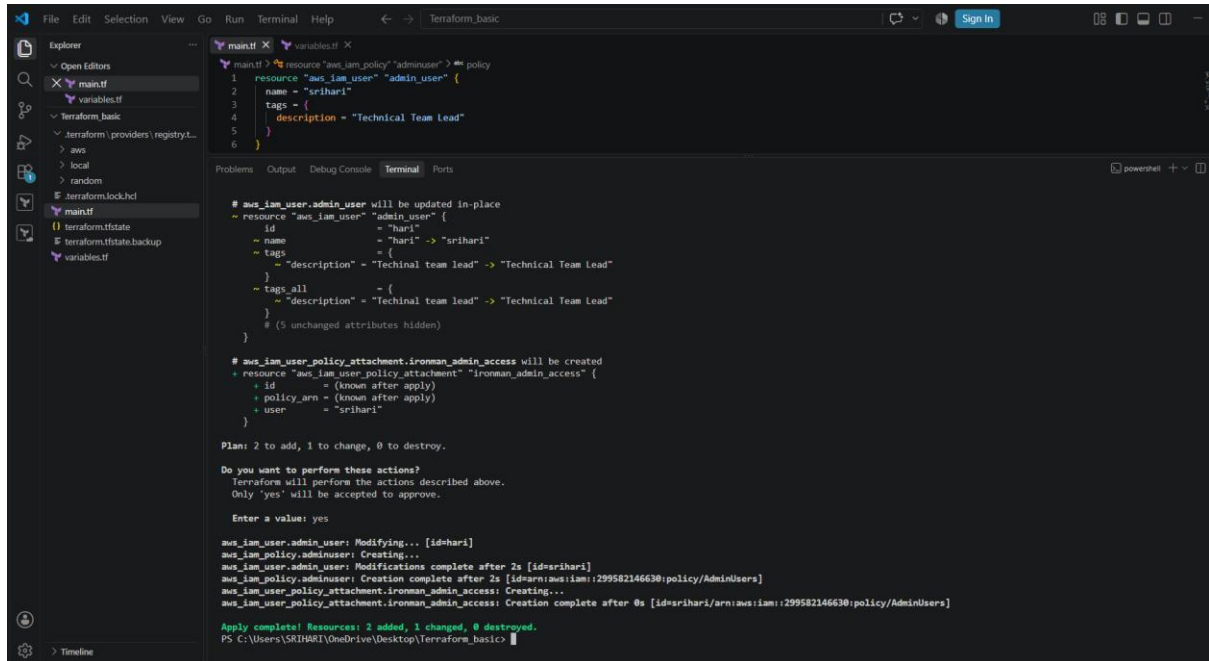
Terraform creates:

IAM Policy: AdminUsers

Policy attachment to IAM user ironman

Output shows:

Apply complete! Resources: 2 added



```
main.tf | variables.tf |  
1 resource "aws_iam_policy" "admin_user" {  
2   name = "srihari"  
3   tags = {  
4     description = "Technical Team Lead"  
5   }  
6 }  
  
# aws_iam_user.admin_user will be updated in-place  
~ resource "aws_iam_user" "admin_user" {  
  ~ id = "hari"  
  ~ name = "hari" -> "srihari"  
  ~ tags = {  
    ~ "description" = "Technical team lead" -> "Technical Team Lead"  
  }  
  ~ tags_all = {  
    ~ "description" = "Technical team lead" -> "Technical Team Lead"  
  }  
  # (5 unchanged attributes hidden)  
}  
  
# aws_iam_user_policy_attachment.ironman_admin_access will be created  
+ resource "aws_iam_user_policy_attachment" "ironman_admin_access" {  
  + id = (known after apply)  
  + policy_arn = (known after apply)  
  + user = "srihari"  
}  
  
Plan: 2 to add, 1 to change, 0 to destroy.  
  
Do you want to perform these actions?  
Terraform will perform the actions described above.  
Only 'yes' will be accepted to approve.  
  
Enter a value: yes  
  
aws_iam_user.admin_user: Modifying... [id:hari]  
aws_iam_policy.admin_user: Creating...  
aws_iam_user.admin_user: Modifications complete after 2s [id:srihari]  
aws_iam_policy.admin_user: Creation complete after 2s [id:arn:aws:iam::299582146630:policy/AdminUsers]  
aws_iam_user_policy_attachment.ironman_admin_access: Creating...  
aws_iam_user_policy_attachment.ironman_admin_access: Creation complete after 0s [id:srihari/arn:aws:iam::299582146630:policy/AdminUsers]  
  
Apply complete! Resources: 2 added, 1 changed, 0 destroyed.  
PS C:\Users\SRIHARI\OneDrive\Desktop\Terraform_basics>
```

Go back to AWS Console

Open:

IAM → Users → srihari → Permissions

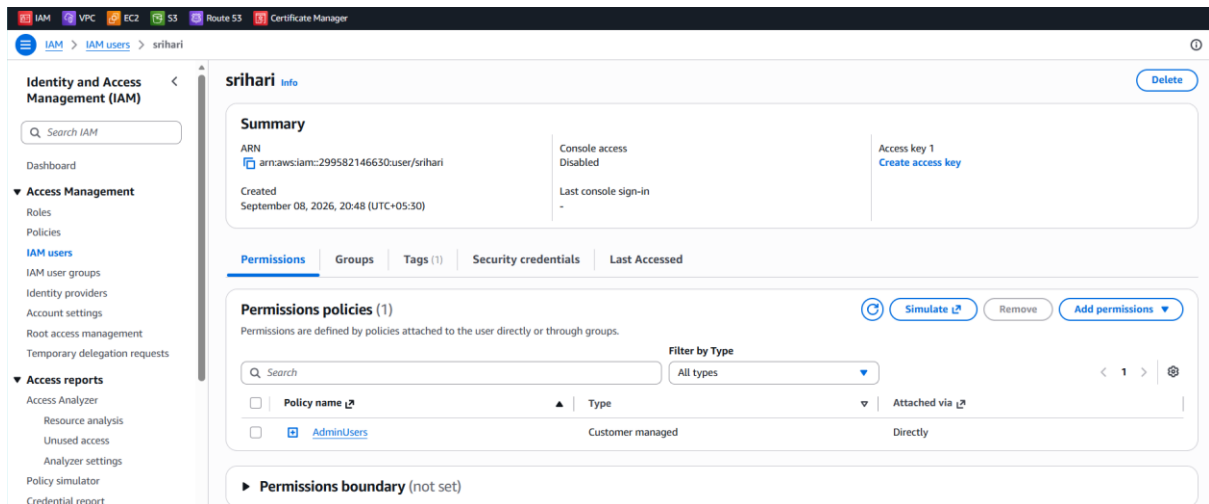
Now you can see:

Permissions policies (1)

Policy name shows as:

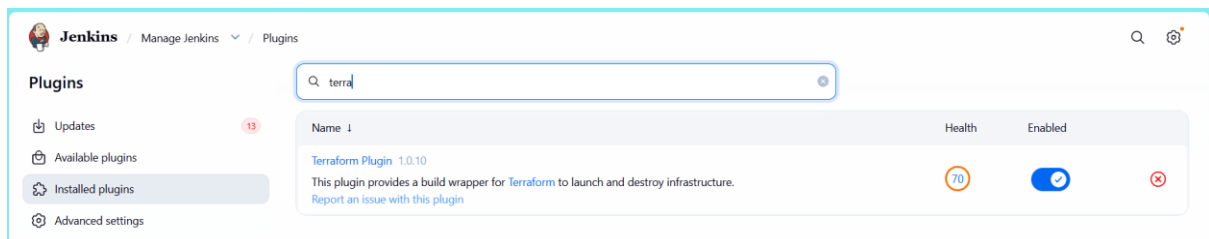
AdminUsers

This confirms **Terraform successfully attached policy to the user**



3. Integrate **Terraform** in Jenkins using the **Terraform plugin**.

- Login to **Jenkins Dashboard**
- Go to: **Manage Jenkins → Plugins**
- Click **Installed plugins / Available plugins**
- Search for **Terraform**
- Install/Enable **Terraform Plugin**
- Confirm plugin status is **Enabled**



- Login to Jenkins server using SSH (root/user)
- Download Terraform binary (zip) inside /opt or required path
- Unzip the terraform package:
- unzip terraform_<version>_linux_amd64.zip
- Move terraform binary to /usr/bin:
- mv terraform /usr/bin/
- Verify installation:
- terraform version
- Confirm terraform path:
- which terraform

- `ls -l /usr/bin/terraform`

```

A newer release of "Amazon Linux" is available.
Version 2023.12.20260817:
Version 2023.12.20260831:
Run "/usr/bin/dnf check-release-update" for full release and version update info

Amazon Linux 2023
https://aws.amazon.com/linux/amazon-linux-2023

Last login: Mon Sep  7 14:55:51 2026 from 18.206.107.29
[ec2-user@ip-172-31-6-115 ~]$ sudo su -
Last login: Mon Sep  7 14:56:05 UTC 2026 on pts/1
[root@ip-172-31-6-115 ~]# terraform -v
Terraform v1.16.0
on linux_amd64

Your version of Terraform is out of date! The latest version
is 1.16.1. You can update by downloading from https://developer.hashicorp.com/terraform/install
[root@ip-172-31-6-115 ~]# find / -name terraform
/var/lib/jenkins/plugins/terraform
/usr/bin/terraform
/usr/share/doc/terraform
[root@ip-172-31-6-115 ~]# ls -ld /usr/bin
dr-xr-xr-x. 2 root root 32768 Sep  2 14:01 /usr/bin
[root@ip-172-31-6-115 ~]# ls -l /usr/bin/terraform
-rwxr-xr-x. 1 root root 120271010 Aug 26 11:23 /usr/bin/terraform
[root@ip-172-31-6-115 ~]#

```

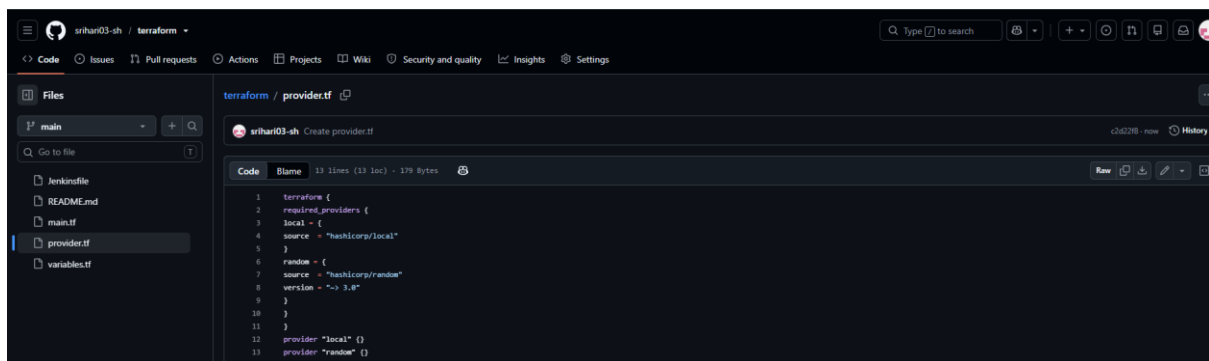
i-00ca6f09e98a51fdb (Jenkins-Server)
PublicIPs: 32.192.178.18 PrivateIPs: 172.31.6.115

- Go to Jenkins Dashboard
- Navigate: **Manage Jenkins** → **Tools**
- Under **Terraform installations**, click **Add Terraform**
- Enter Name: terraform
- Set Install directory as:
- `/usr/bin`

Click **Save / Apply**

The screenshot shows the Jenkins web interface. On the left, a terminal window displays the same Terraform installation commands as the previous image. On the right, the 'Tools - Manage Jenkins' page is open, specifically the 'Terraform installations' section. A new installation named 'terraform' has been added, with the 'Install directory' set to '/usr/bin'. The 'Install automatically' checkbox is unchecked. At the bottom of the page, there are 'Save' and 'Apply' buttons.

- Open your GitHub repo (Terraform project)
- Edit/Create file **provider.tf**
- Add required providers block (example):
- terraform {
- required_providers {
- local = {
- source = "hashicorp/local"
- }
- random = {
- source = "hashicorp/random"
- }
- }
- }
- provider "local" {}
- provider "random" {}
- Commit and push the changes to repository



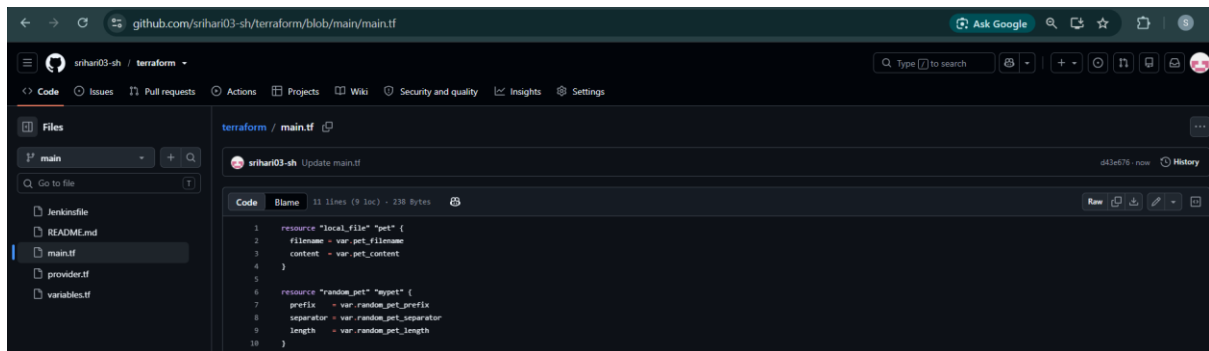
- Open **main.tf** inside repo

Add resources like local_file + random_pet:

```
resource "local_file" "pet" {
  filename = var.pet_filename
  content = var.pet_content
}
```

```
resource "random_pet" "mypet" {
  prefix = var.random_pet_prefix
  separator = var.random_pet_separator
  length = var.random_pet_length
}
```

- Save changes
- Commit and push



- Open **variables.tf** in your Terraform repo
- Define variables with type and defaults:

```
variable "pet_filename" {  
    type = string  
    default = "/tmp/pets.txt"  
}
```

```
variable "pet_content" {  
    type = string  
    default = "my horse is black in colour!"  
}
```

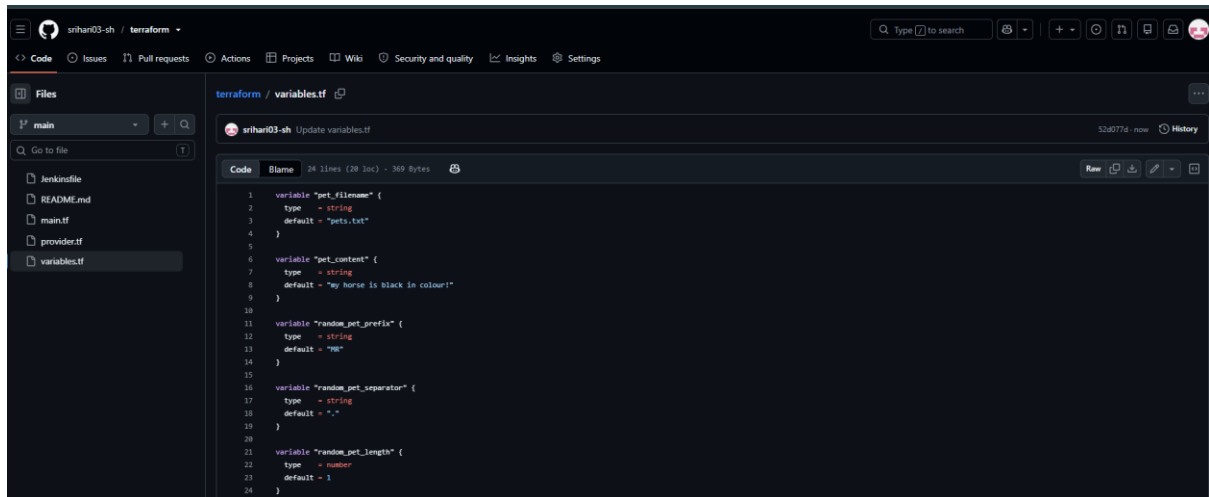
```
variable "random_pet_prefix" {  
    type = string  
    default = "MR"  
}
```

```
variable "random_pet_separator" {  
    type = string  
    default = "."  
}
```

```
variable "random_pet_length" {  
    type = number  
    default = 1
```

}

- Save and commit changes



- Create a file in repo called **Jenkinsfile**
- Add pipeline stages for Terraform:
 1. Checkout
 2. Terraform Init
 3. Terraform Plan
 4. Terraform Apply
- Example Jenkinsfile flow shown in screenshot uses:
 1. terraform init
 2. terraform plan -out=tfplan
 3. terraform apply --auto-approve tfplan
- Commit Jenkinsfile into GitHub

The screenshot shows the Jenkinsfile editor interface. On the left, a 'Files' sidebar lists 'main', 'Jenkinsfile', 'README.md', 'main.tf', 'providers.tf', and 'variables.tf'. The main editor area is titled 'terraform / Jenkinsfile' and shows a code editor with the following content:

```
1 pipeline {
2   agent any
3
4   environment {
5     TF_IN_AUTOMATION = "true"
6   }
7
8   stages {
9
10    stage('Checkout from SCM') {
11      steps {
12        git branch: 'main',
13          url: 'https://github.com/mujahed776/terraform.git'
14      }
15    }
16
17    stage('Terraform Init') {
18      steps {
19        sh '''
20          terraform --version
21          terraform init
22          ...
23        '''
24      }
25    }
26
27    stage('Terraform Plan') {
28      steps {
29        sh '''
30          terraform plan -out=tfplan
31          ...
32        '''
33      }
34    }
35
36    stage('Terraform Apply') {
37      steps {
38        input message: 'Approve Terraform Apply?', ok: 'Apply'
39        sh '''
40          terraform apply -auto-approve tfplan
41        '''
42      }
43    }
44  }
45 }
```

This screenshot shows the continuation of the Jenkinsfile script from the previous image. The code editor displays the following content:

```
13       url: 'https://github.com/mujahed776/terraform.git'
14     }
15   }
16
17   stage('Terraform Init') {
18     steps {
19       sh '''
20         terraform --version
21         terraform init
22         ...
23       '''
24     }
25
26     stage('Terraform Plan') {
27       steps {
28         sh '''
29           terraform plan -out=tfplan
30           ...
31         '''
32       }
33     }
34
35     stage('Terraform Apply') {
36       steps {
37         input message: 'Approve Terraform Apply?', ok: 'Apply'
38         sh '''
39           terraform apply -auto-approve tfplan
40           ...
41         '''
42       }
43     }
44
45     post {
46       success {
47         echo 'Terraform pipeline executed successfully.'
48       }
49       failure {
50         echo 'Terraform pipeline failed.'
51       }
52     }
53   }
54 }
```

- Create a new Jenkins job: **New Item** → **Pipeline**
- Job name: terraform-task-03
- Go to **Configure** → **Pipeline** section

Not secure 32.192.178.18:8080/job/terraform-task-03/configure

Jenkins terraform-task-03 / Configure

Configure

- General
- Triggers
- Pipeline**
- Advanced

Definition

Pipeline script from SCM

SCM ?

Git

Repositories ?

Repository URL ?

https://github.com/srihari03-sh/terraform.git

Credentials ?

- none -

+ Add

Advanced

+ Add Repository

Not secure 32.192.178.18:8080/job/terraform-task-03/configure

Jenkins terraform-task-03 / Configure

Configure

- General
- Triggers
- Pipeline**
- Advanced

Branches to build ?

Branch Specifier (blank for 'any') ?

*/main

+ Add Branch

Repository browser ?

(Auto)

Additional Behaviours

+ Add

Script Path ?

Jenkinsfile

☒ Lightweight checkout ?

Save Apply

Build success

terraform-task-03 - Jenkins

Not secure 32.192.178.18:8080/job/terraform-task-03/

Jenkins terraform-task-03

Status

Changes

Build Now

Delete Pipeline

Configure

Rename

Full Stage View

Stages

Pipeline Syntax

Builds >

Today

#1 3:27 PM

terraform-task-03

Stage View

	Declarative: Checkout SCM	Declarative: Tool Install	Checkout from SCM	Terraform Init	Terraform Plan	Terraform Apply
Average stage times:	408ms	140ms	411ms	1s	698ms	137ms
408ms 140ms 411ms 1s 698ms 137ms	408ms	140ms	411ms	1s	698ms	137ms

408ms

140ms

411ms

1s

698ms

137ms

408ms

140ms

411ms

1s

698ms

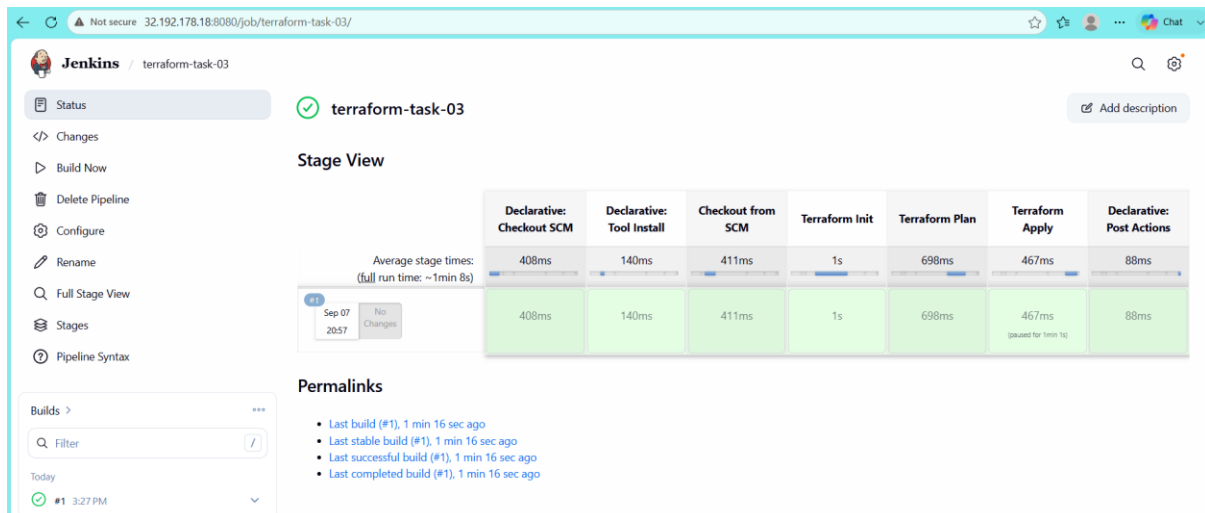
137ms

Approve Terraform Apply?

Apply Abort

137ms (cached for 10s)

Permalinks



4. Create a CI/CD pipeline for a Nodejs Application:

<https://github.com/betawins/Trading-UI.git>

Open Jenkins Dashboard

Go to:

Manage Jenkins → Plugins

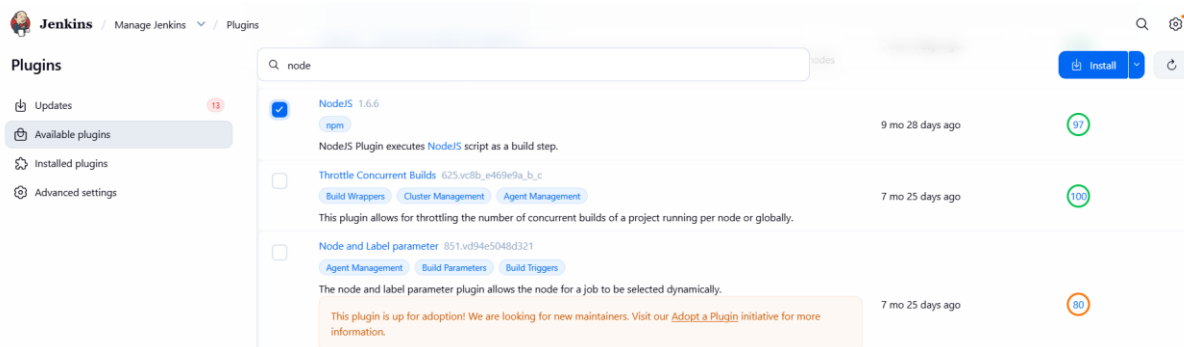
Click **Available plugins**

Search for **nodejs**

Select **NodeJS Plugin**

Click **Install** (top-right button)

Wait until plugin installation is completed



Go to:

Manage Jenkins → Tools

Open **NodeJS** tool configuration again

Select Node version (example shown: **NodeJS 26.8.1**)

Keep Install automatically enabled

Click **Save**

This version will be used in Jenkins pipeline builds

Jenkins / Manage Jenkins / Tools

NodeJS installations ^ Edited

+ Add NodeJS

NodeJS

Name

NodeJS

☒ Install automatically ?

Install from nodejs.org

Version

NodeJS 26.8.1

For the underlying architecture, if available, force the installation of the 32bit package. Otherwise the build will fail

☐ Force 32bit architecture

Global npm packages to install

Specify list of packages to install globally -- see npm install -g. Note that you can fix the packages version by using the syntax "packageName@version"

Global npm packages refresh hours

Save Apply

Open your GitHub repository

Click on **Jenkinsfile**

Add pipeline stages inside Jenkinsfile (Node + npm stages)

```
pipeline {
  agent any
```

```
  stages {
```

```
    stage('Git Checkout') {
      steps {
        git 'https://github.com/betawins/Trading-UI.git'
      }
    }
```

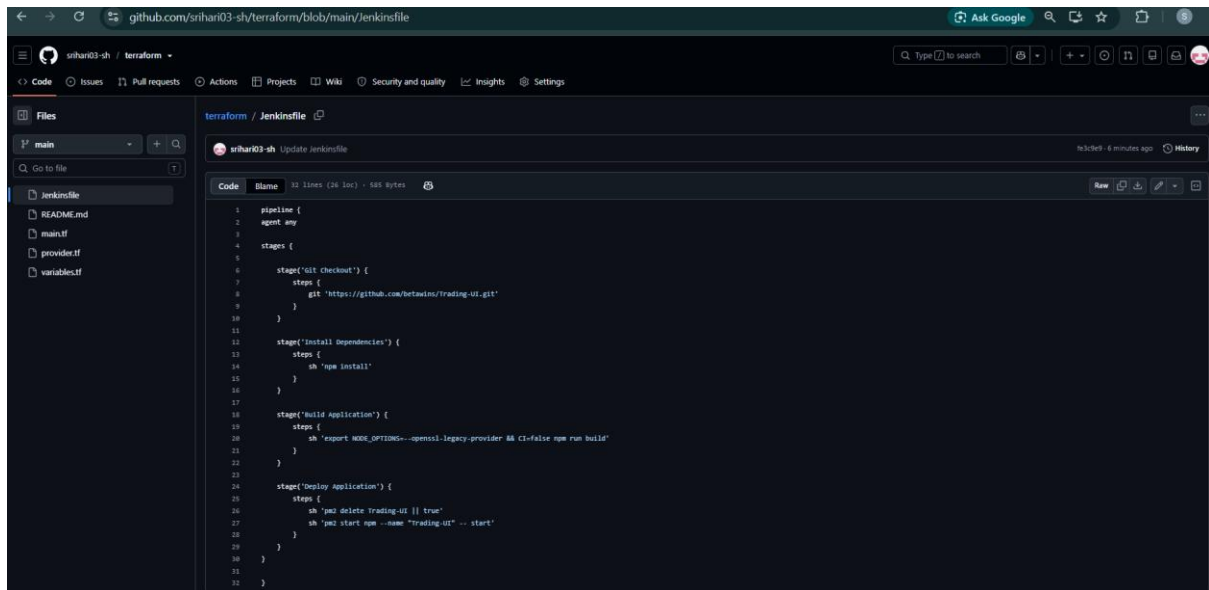
```
    stage('Install Dependencies') {
      steps {
        sh 'npm install'
      }
    }
```

```
    stage('Build Application') {
      steps {
        sh 'export NODE_OPTIONS=--openssl-legacy-provider && CI=false npm run build'
      }
    }
```

```
    stage('Deploy Application') {
      steps {
        sh 'pm2 delete Trading-UI || true'
        sh 'pm2 start npm --name "Trading-UI" -- start'
      }
    }
  }
```

```
}
```

Save the Jenkinsfile and **Commit changes**



Open Jenkins Dashboard

Click **New Item**

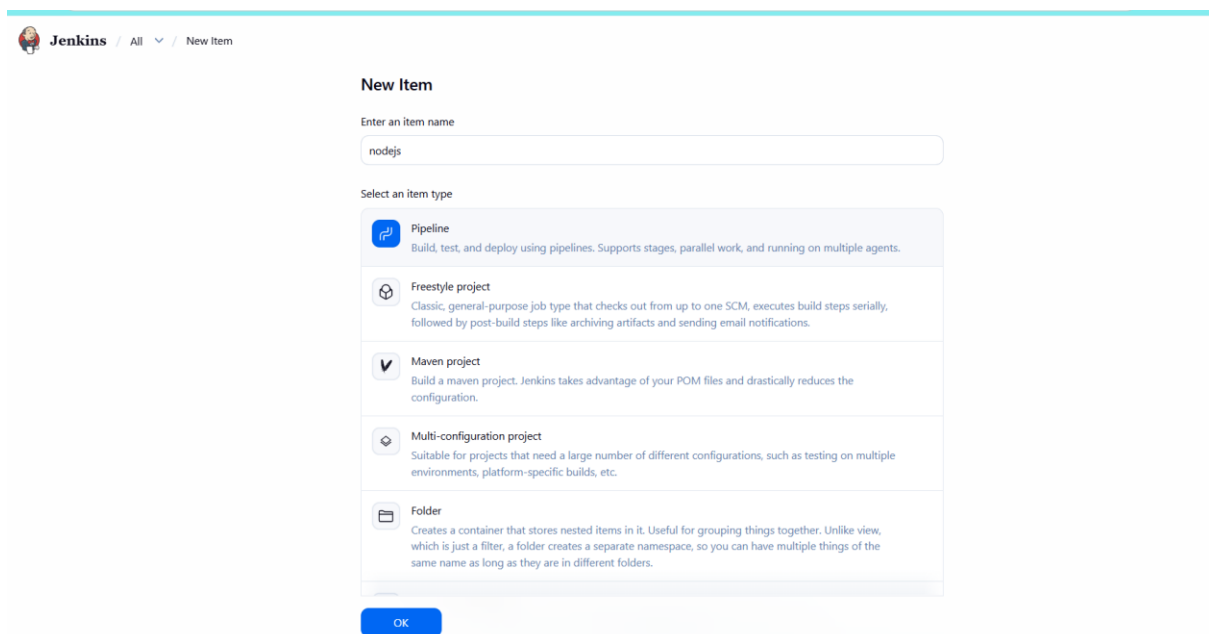
Enter job name:

nodejs

Select:

Pipeline

Click **OK**



Inside job configuration, go to **Pipeline**

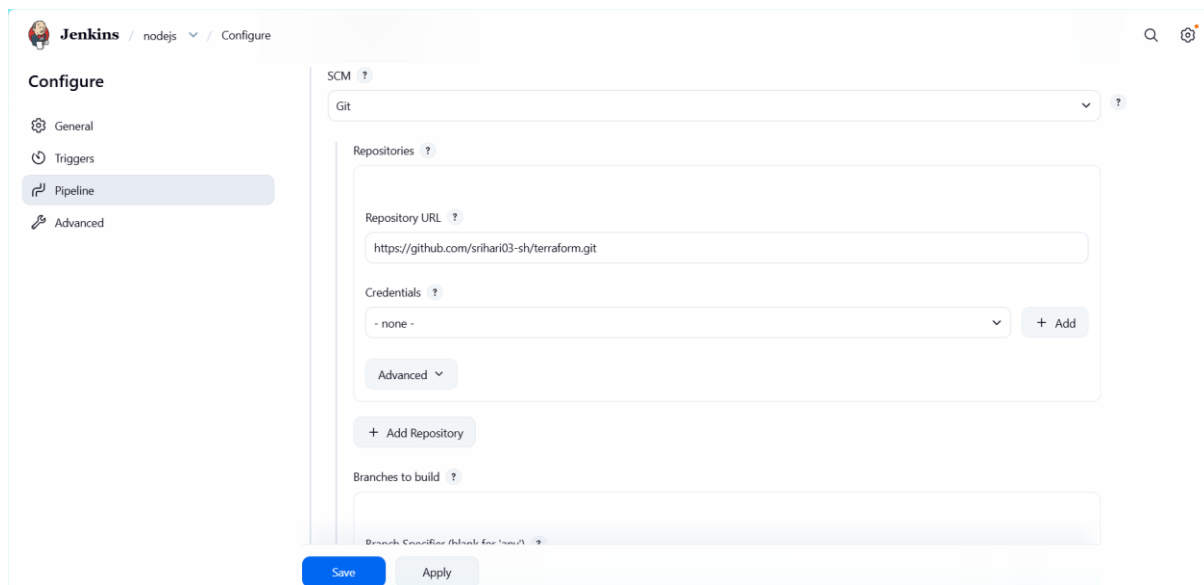
In **Definition**, select:
Pipeline script from SCM

In **SCM**, choose:
Git

Under **Repository URL**, enter GitHub repo link
Example:

<https://github.com/srihari03-sh/terraform.git>

Click **Save**



In the same Pipeline configuration page:
Under **Branches to build**

Add branch:

*/master

(or */main if your repo is main branch)

In **Script Path**, type:

Jenkinsfile

Click **Save / Apply**

Jenkins / nodejs / Configure

Configure

- General
- Triggers
- Pipeline**
- Advanced

Branches to build ?

Branch Specifier (blank for 'any') ?

*/master

+ Add Branch

Repository browser ?

(Auto)

Additional Behaviours

+ Add

Script Path ?

Jenkinsfile

☒ Lightweight checkout ?

Save Apply

Open Jenkins job: nodejs
 Click the build history build number
 Open **Stage View**
 You can see pipeline stages like:

Checkout SCM

Tool Install

Checkout

Check Node & NPM

Install Dependencies

Build

Test

Jenkins / nodejs

nodejs

Stage View

Average stage times: (full run time: ~1min 8s)

	Declarative: Checkout SCM	Git Checkout	Install Dependencies	Build Application	Deploy Application
#16 Sep 09 19:31 No Changes	555ms	259ms	57s	8s	829ms
#15 Sep 09 19:27 2 commits	737ms	272ms	9s	8s	595ms failed
#14 Sep 09 19:26 No Changes					
#13 Sep 09 19:24 1 commit	313ms	307ms	39s	19s failed	50ms failed
#12 Sep 09 19:21 No Changes	314ms	343ms	9s	1s	64ms

Builds >

Filter

Today

- #16 2:01 PM
- #15 1:57 PM
- #14 1:56 PM
- #13 1:54 PM
- #12 1:51 PM

5. Explain **10 Maven commands**.

1) **mvn -version**

Checks Maven version + Java version installed.

2) **mvn clean**

Deletes the old compiled files (removes target/ folder).

3) **mvn compile**

Compiles the source code from src/main/java.

4) **mvn test**

Runs unit test cases from src/test/java.

5) **mvn package**

Builds the project and creates a **JAR/WAR file** inside target/.

6) **mvn install**

Installs the built artifact into **local Maven repo** (~/.m2/repository).

7) **mvn validate**

Checks if the project structure and pom.xml are correct.

8) **mvn clean install**

First cleans the old build, then builds and installs fresh package.

9) **mvn dependency:tree**

Shows dependency hierarchy (which dependency depends on what).

10) **mvn deploy**

Uploads the final package to a **remote repository** (Nexus/Artifactory).